

Deliverable 3: Implementation & Automated Testing

Due: November 9, 2025

In this deliverable, your team will begin building the system based on your approved design from Deliverable II. The goal is to implement approximately half of your system's core requirements and demonstrate basic automated testing to verify correctness.

1. Implementation Scope

- Implement at least 50% of the functional requirements defined in Deliverable I.
- Focus on core features that demonstrate your system's main functionality (e.g., authentication, data processing, communication with backend).
- Each implemented feature should be traced back to a specific requirement ID from Deliverable I.
- Non-functional aspects (like performance or security) should be partially addressed where possible.

2. Automated Testing

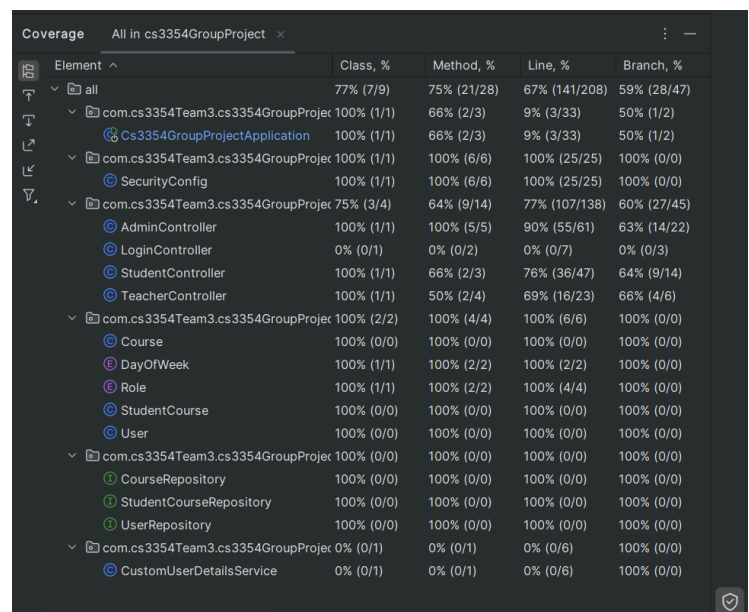
You are required to write and demonstrate automated tests for your implemented features and the system. You should write unit tests, integration tests, and system level tests.

- You are required to **design test cases for all system requirements to ensure full requirement coverage**. However, you should write and demonstrate automated test cases only for the implemented requirements in this deliverable.
- You need to add sufficient test case to achieve a high **code coverage**.
- Each test should correspond to one or more implemented requirement IDs.
- Use a standard testing framework (e.g., pytest, JUnit)

Answer:

To perform testing on our Course Registration System, we implemented a multi-layered or tiered approach. This method of testing involved first starting with unit tests to check for the general functionality of the software. From there, we would implement integration tests before finally conducting system tests.

We started with the unit tests, and many of the functional requirements are able to be verified through this form of automated testing. This allowed us to verify multiple requirements. The unit tests for this project



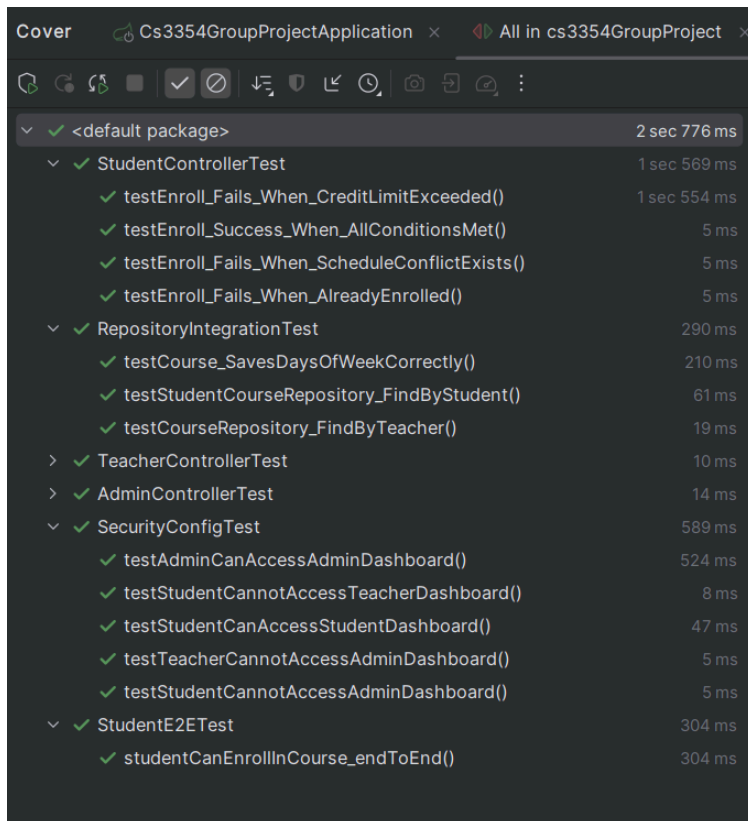
Element	Class, %	Method, %	Line, %	Branch, %
all	77% (7/9)	75% (21/28)	67% (141/208)	59% (28/47)
com.cs3354Team3.cs3354GroupProje	100% (1/1)	66% (2/3)	9% (3/33)	50% (1/2)
Cs3354GroupProjectApplication	100% (1/1)	66% (2/3)	9% (3/33)	50% (1/2)
com.cs3354Team3.cs3354GroupProje	100% (1/1)	100% (6/6)	100% (25/25)	100% (0/0)
SecurityConfig	100% (1/1)	100% (6/6)	100% (25/25)	100% (0/0)
com.cs3354Team3.cs3354GroupProje	75% (3/4)	64% (9/14)	77% (107/138)	60% (27/45)
AdminController	100% (1/1)	100% (5/5)	90% (55/61)	63% (14/22)
LoginController	0% (0/1)	0% (0/2)	0% (0/7)	0% (0/3)
StudentController	100% (1/1)	66% (2/3)	76% (36/47)	64% (9/14)
TeacherController	100% (1/1)	50% (2/4)	69% (16/23)	66% (4/6)
com.cs3354Team3.cs3354GroupProje	100% (2/2)	100% (4/4)	100% (6/6)	100% (0/0)
Course	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
DayOfWeek	100% (1/1)	100% (2/2)	100% (2/2)	100% (0/0)
Role	100% (1/1)	100% (2/2)	100% (4/4)	100% (0/0)
StudentCourse	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
User	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
com.cs3354Team3.cs3354GroupProje	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
CourseRepository	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
StudentCourseRepository	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
UserRepository	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
com.cs3354Team3.cs3354GroupProje	0% (0/1)	0% (0/1)	0% (0/6)	100% (0/0)
CustomUserDetailsService	0% (0/1)	0% (0/1)	0% (0/6)	100% (0/0)

were the StudentControllerTest, TeacherControllerTest, and AdminControllerTest.

Moving from the unit tests, next we wrote and ran integration tests. These checks ensure that some of the units will be able to work with one another. These were the SecurityConfigTest and RepositoryIntegrationTest. Lastly, we ran a small end-to-end test to perform a large system-level evaluation of our code. All of these tests ran, and the code base itself had close to 70% coverage.

Requirement	Description	Test Level	Test Class/ Methods
FR#1	Users can log in with their unique username and password	Integration, System	SecurityConfigTest, StudentE2ETest
FR#2	Professors can submit a course to teach	Unit, Integration	TeacherControllerTest, RepositoryIntegrationTest
FR#3	Professors can provide course information	Unit, Integration	TeacherControllerTest, RepositoryIntegrationTest
FR#4	The system will track students previous courses	Not Tested	
FR#5	Students can search for courses and ADD or DROP classes	Unit, Integration, System	StudentControllerTest, StudentE2ETest, RepositoryIntegrationTest
FR#6	The system will check prerequisites before allowing registration	Not Tested	
FR#7	Students cannot register for more than the maximum credit hours	Unit	StudentControllerTest
FR#8	Students cannot register for classes at the same time	Unit	StudentControllerTest
FR#9	Students cannot register for a course if the class is full	Not Tested	
FR#10	Students and waitlist a course and will be notified when available	Not Tested	
FR#11	Students cannot register before their designated time	Not Tested	

FR#12	Students cannot drop classes after the deadline	Not Tested	
FR#13	The system allows Admin to create classes	Unit	AdminControllerTest
FR#14	Admins can add or drop classes from a student's schedule	Unit	AdminControllerTest
NFR#1	The system will be secure through username and password authentication	Integration, System	SecurityConfigTest, StudentE2ETest
NFR#2	The system shall be readily available 99% of the time	Not Tested	
NFR#3	Users can login within 10 seconds of entering credentials	Not Tested	
NFR#4	The system will only allow certain users to view students' data	Unit, Integration	SecurityConfigTest, StudentControllerTest



Cover	Cs3354GroupProjectApplication	All in cs3354GroupProject
<default package>	2 sec 776 ms	
StudentControllerTest	1 sec 569 ms	
testEnroll_Fails_When_CreditLimitExceeded()	1 sec 554 ms	
testEnroll_Success_When_AllConditionsMet()	5 ms	
testEnroll_Fails_When_ScheduleConflictExists()	5 ms	
testEnroll_Fails_When_AlreadyEnrolled()	5 ms	
RepositoryIntegrationTest	290 ms	
testCourse_SavesDaysOfWeekCorrectly()	210 ms	
testStudentCourseRepository_FindByStudent()	61 ms	
testCourseRepository_FindByTeacher()	19 ms	
TeacherControllerTest	10 ms	
AdminControllerTest	14 ms	
SecurityConfigTest	589 ms	
testAdminCanAccessAdminDashboard()	524 ms	
testStudentCannotAccessTeacherDashboard()	8 ms	
testStudentCanAccessStudentDashboard()	47 ms	
testTeacherCannotAccessAdminDashboard()	5 ms	
testStudentCannotAccessAdminDashboard()	5 ms	
StudentE2ETest	304 ms	
studentCanEnrollInCourse_endToEnd()	304 ms	

Of the 20 combined functional and non-functional tests, we implemented and tested to verify 10 to meet the 50% minimum required. We focused on the main requirements that were needed for the system to have basic functionality (login, add classes, etc.) to be able to verify that the foundation was solid. Moving from here, we will develop other features and aspects (waitlists, registration times, etc.) to complete the other requirements.

3. Documentation Report PDF

Include a single document summarizing the development and testing process:

- Overview

Briefly describe the system's purpose, main functionalities, and the problem it solves. Clearly explain how the work in Deliverable III builds upon Deliverables I and II, showing that earlier artifacts (requirements, design diagrams, and models) were actively used to guide implementation.

Answer:

Our system is a university course registration system with the purpose of allowing students of the university to register for classes. The system is also built to allow professors to upload the courses they will teach, so that students can sign up for them. Our system also allows system administrators to perform these tasks on behalf of students and teachers, as well as perform other administrative duties. Our system reflects a lot of the work from the previous deliverables, as the project was built based on the models and requirements created. For example, we essentially went down the list of functional requirements from Deliverable I to implement most of the main features into the system, such as allowing users to log in using their unique email and password (functional requirement 1), allowing students to enroll or drop classes to fit their schedule, following a maximum credit requirement (functional requirements 5, 7, and 8). We also used many of the diagrams we created, such as the activity, state, and class diagrams from Deliverable II, to help build our project. While we did not end up building our system exactly like our models due to time constraints and because we tried to simplify our system to be more user-friendly, our models from Deliverable II helped us have a foundation for what our system was supposed to look like. We created our user classes/controllers based on the classes from our class diagram and designed the general flow of our system based on the activity and state diagram. We also used the MVC pattern discussed in deliverable 2 in our project, where most of our user and repository files serve as the model, most of our front-end HTML files serve as the view, and most of the files ending in controller serve as the controller.

- Development Process:

Explain the development approach (Agile sprints, milestones, or Plan-Driven phases) and how the team utilized previous deliverables to plan and execute this phase. Describe how requirements and design artifacts were referenced during implementation, and how the team coordinated tasks to ensure traceability to the original specifications.

Answer:

We knew that there were going to be a few changes to our plan due to only having a short period of time to work on the project and not having much experience with web development, which is why we went with a more agile approach, but we also used some aspects of a plan-driven approach. Since we were modeling our system based on other university registration systems, and this wasn't going to be a new idea, we already knew a lot of the requirements and features that we were going to try to implement, similar to a plan-driven approach. However, we also knew we were not gonna have much time to develop the actual system due to classes and lack of experience, which is where we used more agile methods of implementation. During the first half of the semester, we started with smaller sprints, trying to get the base level of our system set up, and also used short week-long sprints for each deliverable. Once we had most of the system designed and the requirements engineering done, we were able to use extreme programming to complete over 50% of our requirements in just a few sprints. By already having most of the system design done, we were able to split up the work amongst ourselves so that some people were working on the frontend, backend, or writing tests. By referencing deliverables 1 and 2, we were able to spend less time asking each other small questions on the design of the system and focus more time on implementing the features specified during our requirements engineering. Also, since some people were focusing more on writing tests based on the requirements we created, we were able to make sure that the system we developed followed the requirements and design we came up with.

- Design and Implementation Summary:
Summarize the specific features or requirements developed in this deliverable, referencing their corresponding requirement IDs from Deliverable I. Describe how the implementation followed the design artifacts from Deliverable II, including any adjustments made. Mention the programming language(s), frameworks, and tools used, and provide details of the environment setup.

Answer:

Since most of us had more experience using Java, we ended up using Spring Boot to help build our project. Spring Boot allowed us to create our entire project with mostly Java and some HTML and CSS for our front end. We also chose to use Spring Boot because it handles many things for the user, such as HTTP request methods and integrating a MySQL database. This allowed us to focus more on implementing features quickly. Some of the main requirements we developed include allowing users to log in to the system using their unique email and passwords (FR#1), allowing students to add and drop classes to their schedule as long as they do not go over the max credit limit (FR#5, 7, 8), allowing professors to create and delete courses for students to register for and include basic information about the course (FR#2, 3), and allow system

administrators to perform these tasks on behalf of students and professors (FR#13, 14). As stated before, we also used a lot of the diagrams we created, like the activity, state, and class diagrams from deliverable 2, to help build our project. There were some features that we have not included from Deliverables I and II because of time constraints, and because some of them made the system more complicated. For example, in Deliverable II, we designed our system to have multiple pages and steps for our users to interact with the system, but we simplified the number of pages that users will navigate through so that students can register, drop, and view their courses all on the same page. Again, we also used the MVC pattern discussed in Deliverable II in our project, where most of our user and repository files serve as the model, most of our front-end HTML files serve as the view, and most of the files ending in controller serve as the controller.

- **Development Environment:**
Provide detailed information on the programming language(s) and their versions, libraries, frameworks, and tools used. Include environment configuration details such as IDEs, operating systems, database versions, or any other dependencies required to run and test the system successfully.

Answer:

Most of our backend and logic was built using Java JDK 25, and for our frontend, we used HTML with CSS and JavaScript. We also used Spring Boot 3.5.7 as our framework, MySQL for our database, and Maven 1.5.20 to help build our project. These dependencies are also listed in the "pom.xml" file. Each of us used our own IDEs, but most of us used IntelliJ.

- **Testing Summary:**
Explain how the implemented features were tested. Include details on the testing framework (e.g., pytest, JUnit, Mocha, etc.), the type of tests conducted (unit, integration, or system), and how each test case traces back to one or more requirements. Summarize the achieved code coverage and describe how it demonstrates sufficient testing of all implemented requirements.

Answer:

Our testing summary, along with the type of tests conducted and how each test aligns with our requirements, can be found in part 2 of this deliverable. Essentially, we conducted unit and system tests to ensure that our system complies with each of the requirements we listed in deliverable 1. Code coverage and other statistics are also listed above.

- **Challenges Faced:**
Discuss major technical or teamwork challenges encountered during implementation and testing, and how they were resolved.

Answer:

A lot of the technical challenges that we faced were in setting up to actually be able to develop and test the system. Since many of us lack experience in web development and database systems, we had many questions and got confused about how the overall system works in general. However, we answered our questions through research, trial and error, and just helping one another. Once all of us were set up, we started developing and working together to implement the requirements. We had minimal issues in agreeing on the design of the system. For example, for the frontend, each of us had different preferences on how the UI should be structured and designed.

- **Design Assumptions & Revisions:**
Identify which initial design assumptions held true and which required revision during development.

Answer:

One of the initial designs that stayed true is the architectural pattern for our project, which is MVC. Model being our repository files, the front end being the view, and our files that end in controller are the controller. A lot of change in design comes down to how the user interacts with the user. Originally, we had every feature in its own dedicated page. Such as the schedule for the student, the teacher adding a class, or a student just searching for a class. We then decided that this was much too difficult to implement and maintain. It would be much easier to implement fewer pages because it would be much easier to maintain, implement, and organize.

- **Impact of Automated Testing:**
Explain how automated testing improved the quality, reliability, or maintainability of your system.

Answer: There's so much that we as humans can consider testing on. More, since we're students and have experience with using class registration systems, there are some things we don't even consider testing. Because of this, automated testing greatly helped with improving the maintainability and reliability of the system. Now our system is able to handle any scenario it may be in. This is very important because many students have very busy schedules depending on their degree plan. This was solved by making the overall system much simpler, so there were fewer things we all needed to agree upon and to start implementation faster.

- **Screenshots & Evidence:**

Include screenshots, logs, or test reports that demonstrate successful feature execution and passing test results.

Answer:

Coverage All in cs3354GroupProject				
Element	Class, %	Method, %	Line, %	Branch, %
all	77% (7/9)	75% (21/28)	67% (141/208)	59% (28/47)
com.cs3354Team3.cs3354GroupProje	100% (1/1)	66% (2/3)	9% (3/33)	50% (1/2)
Cs3354GroupProjectApplication	100% (1/1)	66% (2/3)	9% (3/33)	50% (1/2)
com.cs3354Team3.cs3354GroupProje	100% (1/1)	100% (6/6)	100% (25/25)	100% (0/0)
SecurityConfig	100% (1/1)	100% (6/6)	100% (25/25)	100% (0/0)
com.cs3354Team3.cs3354GroupProje	75% (3/4)	64% (9/14)	77% (107/138)	60% (27/45)
AdminController	100% (1/1)	100% (5/5)	90% (55/61)	63% (14/22)
LoginController	0% (0/1)	0% (0/2)	0% (0/7)	0% (0/3)
StudentController	100% (1/1)	66% (2/3)	76% (36/47)	64% (9/14)
TeacherController	100% (1/1)	50% (2/4)	69% (16/23)	66% (4/6)
com.cs3354Team3.cs3354GroupProje	100% (2/2)	100% (4/4)	100% (6/6)	100% (0/0)
Course	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
DayOfWeek	100% (1/1)	100% (2/2)	100% (2/2)	100% (0/0)
Role	100% (1/1)	100% (2/2)	100% (4/4)	100% (0/0)
StudentCourse	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
User	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
com.cs3354Team3.cs3354GroupProje	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
CourseRepository	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
StudentCourseRepository	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
UserRepository	100% (0/0)	100% (0/0)	100% (0/0)	100% (0/0)
com.cs3354Team3.cs3354GroupProje	0% (0/1)	0% (0/1)	0% (0/6)	100% (0/0)
CustomUserDetailsService	0% (0/1)	0% (0/1)	0% (0/6)	100% (0/0)

Cover Cs3354GroupProjectApplication All in cs3354GroupProject	
<default package>	2 sec 776 ms
StudentControllerTest	1 sec 569 ms
testEnroll_Fails_When_CreditLimitExceeded()	1 sec 554 ms
testEnroll_Success_When_AllConditionsMet()	5 ms
testEnroll_Fails_When_ScheduleConflictExists()	5 ms
testEnroll_Fails_When_AlreadyEnrolled()	5 ms
RepositoryIntegrationTest	290 ms
testCourse_SavesDaysOfWeekCorrectly()	210 ms
testStudentCourseRepository_FindByStudent()	61 ms
testCourseRepository_FindByTeacher()	19 ms
TeacherControllerTest	10 ms
AdminControllerTest	14 ms
SecurityConfigTest	589 ms
testAdminCanAccessAdminDashboard()	524 ms
testStudentCannotAccessTeacherDashboard()	8 ms
testStudentCanAccessStudentDashboard()	47 ms
testTeacherCannotAccessAdminDashboard()	5 ms
testStudentCannotAccessAdminDashboard()	5 ms
StudentE2ETest	304 ms
studentCanEnrollInCourse_endToEnd()	304 ms